AZURE CLI RK2 LAB

1- **VNet & Subnet:** Kubernetes node'ları birbirleriyle dış internet (Public IP) üzerinden **konuşmaz**. Yüksek hız, düşük gecikme ve güvenlik için tüm kubelet, etcd ve pod-to-pod (VXLAN) trafiği yerel özel ağ (10.10.1.0/24) üzerinden akar.

- **Sadece 2 Public IP Kullanımı:** Veritabanı ve Redis sunucularının internete doğrudan açık bir IP'si olması güvenlik açığıdır ve Azure'da her Standard Public IP saatlik ücret yazar. Sadece vm-master (SSH kapısı / Bastion görevi görür) ve vm-worker-app (kullanıcıların web sitene gireceği kapı) Public IP alır.

Tüm yapıyı tek seferde silebilmek için önce bir Resource Group, ardından VNet ve NSG oluştur:

1. Resource Group
az group create --name rg-rke2-manual --location denmarkeast

2. VNet ve Subnet (10.10.1.0/24)
az network vnet create \
  --resource-group rg-rke2-manual \
  --name vnet-rke2 \
  --address-prefix 10.10.0.0/16 \
  --subnet-name snet-nodes \
  --subnet-prefix 10.10.1.0/24

3. Network Security Group (NSG)
az network nsg create --resource-group rg-rke2-manual --name nsg-rke2

NSG Kuralları (Dışarıdan Gelen Trafik İçin): Azure NSG varsayılan olarak **aynı VNet içindeki (VirtualNetwork tag'i)** tüm makinelerin birbirleriyle tüm portlardan (9345, 6443, 10250, 8472/UDP) konuşmasına izin verir (AllowVnetInBound). Senin sadece dış dünyadan gelecek SSH (22) ve HTTP/HTTPS (80/443) izinlerini eklemen gerekir:

$4 SSH (Port 22) ve HTTP/HTTPS (80, 443) İzinleri

az network nsg rule create -g rg-rke2-manual --nsg-name nsg-rke2 -n Allow-SSH --priority 100 --destination-port-ranges 22 --access Allow --protocol Tcp
az network nsg rule create -g rg-rke2-manual --nsg-name nsg-rke2 -n Allow-Web --priority 110 --destination-port-ranges 80 443 --access Allow --protocol Tcp

5NSG'yi Subnet'e Bağla (Böylece her VM'e tek tek NSG bağlamakla uğraşmazsın)

az network vnet subnet update -g rg-rke2-manual --vnet-name vnet-rke2 -n snet-nodes --network-security-group nsg-rke2

1. İki Farklı Bağlama Yöntemi (NIC vs. Subnet)

Plaintext

```
YÖNTEM A: NIC (VM Ağ Kartı) Seviyesinde NSG (Ayrı Ayrı)
[Subnet: snet-nodes]
   ├── [NSG-1] ──> [VM-Master NIC]
   ├── [NSG-2] ──> [VM-Worker-PG NIC]
   └── [NSG-3] ──> [VM-Worker-App NIC]

YÖNTEM B: Subnet Seviyesinde NSG (Tek Merkezden - Bizim Yaptığımız)
[NSG: nsg-rke2] ──> [Subnet: snet-nodes]
                      ├── [VM-Master NIC (NSG Yok)]
                      ├── [VM-Worker-PG NIC (NSG Yok)]
                      └── [VM-Worker-App NIC (NSG Yok)]
```

- **NIC (Sanal Ağ Kartı) Seviyesinde:** Azure Portal'dan elle VM oluştururken Azure varsayılan olarak her VM'in ağ kartına (`NIC`) o VM'in adıyla ayrı bir NSG oluşturup bağlar (`vm-master-nsg` , `vm-worker-nsg` gibi). 5 makinen olduğunda 5 ayrı NSG yönetmek zorunda kalırsın.
- **Subnet (Alt Ağ) Seviyesinde:** NSG'yi doğrudan `snet-nodes` alt ağına bağladığında, o subnet'in içine koyduğun **tüm sanal makineler otomatik olarak o NSG'nin kurallarına tabi olur.**

2- VM KURULUMLARI

buna public ip veriyoruz docker img vs çekmesi için lab da vs işimiz rahat olsun diye

- **Production'da:** Hiçbir sunucuya (Master ve App dahil) doğrudan Public IP verilmez. Dışarıdan gelen trafik **Azure Application Gateway / Load Balancer** tarafından karşılanır; dışarıya çıkan trafik ise **Azure NAT Gateway** üzerinden tek bir çıkış IP'siyle (SNAT) çıkar

#1 MASTER NODE

```
az vm create \
  --resource-group rg-rke2-manual \
  --name vm-master \
  --image Ubuntu2404 \
  --size Standard_B2s_v2 \
  --vnet-name vnet-rke2 --subnet snet-nodes \
  --nsg "" \
  --public-ip-sku Standard \
  --storage-sku StandardSSD_LRS \
  --admin-username ysf \
  --generate-ssh-keys
```

#WORKER NODES

public ipleri yok bu sefer

Peki Public IP'si Olmayan Bu Worker'lara Nasıl Bağlanacaksın?

Tam da bir önceki mesajda bahsettiğimiz **Jump Host (Bastion)** senaryosu devreye giriyor:

1. Sadece **`vm-master`** sunucusunun Public IP'si var.
2. Termius'a (veya SSH istemcine) bağlanırken önce `vm-master` sunucusuna girersin.
3. Diğer worker'lara (`vm-worker-infra`, `vm-worker-pg` vb.) bağlanmak istediğinde, Termius içinden `vm-master`'ı **Jump Host** olarak seçersin ve worker'ların sadece **Private IP** adreslerini (`10.10.1.5`, `10.10.1.6` vb.) yazarak doğrudan iç ağdan bağlanırsın.

```
for worker in vm-worker-infra vm-worker-pg vm-worker-mongo vm-worker-app; do
  az vm create \
    --resource-group rg-rke2-manual \
    --name $worker \
    --image Ubuntu2404 \
    --size Standard_B1ms \
    --vnet-name vnet-rke2 --subnet snet-nodes \
    --nsg "" \
    --public-ip-address "" \
    --storage-sku StandardSSD_LRS \
    --admin-username ysf \
    --generate-ssh-keys
done
```

3-KURULUM SONRASI WORKERLARI BAĞLAMA RKE2 KURULUMU

az vm list-ip-addresses -g rg-rke2-manual -o table

VirtualMachine PublicIPAddresses PrivateIPAddresses

---------- ---------- ----------

vm-master 9.205.30.201 10.10.1.4

vm-worker-app 10.10.1.8

vm-worker-infra 10.10.1.5

vm-worker-mongo 10.10.1.7

vm-worker-pg 10.10.1.6

1. Neden RKE2 ve Agent Token Mantığı?

RKE2 (Rancher Kubernetes Engine 2), kubeadm gibi onlarca ön paket (containerd, crictl, cni eklentileri) kurmanı gerektirmez; tamamını tek bir güvenli paket içinde getirir.

- **9345 Portu:** RKE2'nin kendi yönetim (Supervisor) portudur. Worker'lar ilk katılırken node-token ile bu porta başvurur, TLS sertifikalarını alır ve ardından 6443 (Kubernetes API) portundan haberleşmeye başlar.

2. Adım 1: Master (vm-master) Kurulumu

vm-master Public IP'sine SSH ile bağlan (ssh ysf@<MASTER_PUBLIC_IP>):

1. Swap'ı kapat (Kubernetes Kubelet bellek hesaplamasını doğru yapmak için swap istemez)
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab

2. RKE2 Server (Control Plane) indir, etkinleştir ve başlat
curl -sfL https://get.rke2.io | sudo sh -
sudo systemctl enable rke2-server.service
sudo systemctl start rke2-server.service

Master üzerinde kubectl ve node-token ayarları: RKE2 varsayılan olarak kubectl aracını /var/lib/rancher/rke2/bin içine, yetki dosyasını (rke2.yaml) ise /etc/rancher/rke2/ içine kurar. Normal kullanıcınla (ysf) doğrudan kubectl çalıştırabilmek için:

Kubeconfig'i kendi kullanıcına kopyala
mkdir -p ~/.kube
sudo cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config

RKE2'nin kendi kubectl binary'sini PATH'e ekle
echo 'export PATH=$PATH:/var/lib/rancher/rke2/bin' >> ~/.bashrc
source ~/.bashrc

Kontrol et (vm-master Ready görünmeli):
kubectl get nodes
Worker'ların Bağlanması İçin Gereken 2 Bilgiyi Al:

1. Master'ın Private IP adresi (Örn: 10.10.1.4):
hostname -I | awk '{print $1}'

2. Gizli Agent Kayıt Token'ı (K10... ile başlayan uzun string):
sudo cat /var/lib/rancher/rke2/server/node-token

PUBLIC API İLE NAT GATEWAY KURULUMU SADECE İÇERİDEN DIŞARIYA TRAFİĞE İZİN VERİR

1. NAT Gateway için Public IP oluştur (2/3 IP kullanılmış olacak)
az network public-ip create -g rg-rke2-manual -n pip-nat-gw --sku Standard --zone 1 2 3

2. NAT Gateway oluştur
az network nat gateway create -g rg-rke2-manual -n nat-gw-rke2 --public-ip-addresses pip-nat-gw --idle-timeout 10

3. Subnet'e bağla
az network vnet subnet update -g rg-rke2-manual --vnet-name vnet-rke2 -n snet-nodes --nat-gateway nat-gw-rke2

İleride Ingress kurduğunda web sitene tarayıcıdan girebilmen için son IP hakkını `vm-worker-app` sunucusuna ver:

1. App Worker için Public IP oluştur (3/3 IP kotası tam dolacak)
az network public-ip create -g rg-rke2-manual -n pip-worker-app --sku Standard --zone 1 2 3

2. vm-worker-app ağ kartına bağla
az network nic ip-config update -g rg-rke2-manual --nic-name vm-worker-appVMNic -n ipconfigvm-worker-app --public-ip-address pip-worker-app

3. Adım 2: 4 Worker (vm-worker-*) Kurulumu ve Master'a Bağlama

Her bir worker sunucusuna SSH ile bağlan ve aşağıdaki işlemleri uygula:

Eğer bilgisayarındaki SSH anahtarı doğrudan tanınıyorsa ve kullanıcı adları her iki makinede de aynıysa (`ysf`), sadece şu şekilde de yazabilirsin:

ssh -J ysf@9.205.30.201 10.10.1.8 workerlara bağlanma

1. Swap'ı kapat
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab

2. RKE2'yi "agent" (Worker) modunda kur
RKE2 agent yükleyicisini doğru yetkiyle çalıştır:
curl -sfL https://get.rke2.io | sudo INSTALL_RKE2_TYPE="agent" sh -

3. Konfigürasyon klasörünü oluştur ve Master bağlantısını yaz
sudo mkdir -p /etc/rancher/rke2/
cat <<EOF | sudo tee /etc/rancher/rke2/config.yaml

server: https://10.10.1.4:9345
token:
K102c1696e85a9c8b8600f9a91013dfae2bce82af0b43a30b9548d252a89bf3bba0::server:e8
08cb83149e01b580d3f06b97a23c77
EOF

4. Agent servisini başlat
sudo systemctl enable rke2-agent.service
sudo systemctl start rke2-agent.service

4 worker'da da bu işlemi yaptıktan yaklaşık 1-2 dakika sonra Master terminalinden kontrol et

kubectl get nodes -o wide
Beş sunucunun tamamını (vm-master ve 4 adet vm-worker-*) **Ready** olarak göreceksin.

Node Rollerini (Labels), İzolasyonu (Taints) ve Depolamayı Ayarlama

1. Neden Label ve Taint Birlikte Kullanılır?

Çok karıştırılan bir noktadır:

- **Label + nodeSelector (Mıknatıs):** *"PostgreSQL pod'u sadece tier=db-sql etiketi olan sunucuya gitsin"* demektir. Ancak bu işlem, **başka pod'ların** (örneğin normal bir web pod'unun) gelip PostgreSQL sunucusuna yerleşmesini **engellemez!**
- **Taint + Toleration (Kalkan):** *"Bu sunucuda dedicated=db-sql:NoSchedule lekesi (taint) var; üzerinde özel izni (toleration) olmayan hiçbir yabancı pod buraya adım atamaz"* demektir.
- Gerçek dünyada kritik sunucuları izole etmek için **ikisi birlikte** kullanılır.

2. Etiketleme ve Taint Komutları (vm-master üzerinde)

Bash
1. Master sunucusuna normal iş yükü pod'larının yerleşmesini engelle:
kubectl taint nodes vm-master CriticalAddonsOnly=true:NoSchedule

2. Infra (Redis) Worker:
kubectl label nodes vm-worker-infra tier=infra
kubectl taint nodes vm-worker-infra dedicated=infra:NoSchedule

3. PostgreSQL Worker:
kubectl label nodes vm-worker-pg tier=db-sql

kubectl taint nodes vm-worker-pg dedicated=db-sql:NoSchedule

4. MongoDB Worker:
kubectl label nodes vm-worker-mongo tier=db-nosql
kubectl taint nodes vm-worker-mongo dedicated=db-nosql:NoSchedule

5. App Worker (Taint koymuyoruz ki Ingress ve genel podlar rahatça yerleşsin):
kubectl label nodes vm-worker-app tier=app

3. Kalıcı Depolama (StorageClass - Local Path Provisioner)

Neden Yapıyoruz? Bitnami Helm chart'ları ile PostgreSQL, MongoDB veya Redis kurduğunda, bu chart'lar otomatik olarak bir PersistentVolumeClaim (PVC) oluşturur. Yönetilen bir bulut servisi (AKS) kullanmadığın için kümede PVC'ye otomatik disk kesecek bir StorageClass yoktur ve pod'ların sonsuza kadar **Pending** durumunda kalır. Rancher'ın local-path-provisioner aracı, PVC talebi geldiğinde ilgili worker node'un kendi diski üzerinde (/opt/local-path-provisioner) otomatik klasör açıp PV olarak bağlar.

Bash
1. Rancher Local Path Provisioner kur:
kubectl apply -f
https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.28/deploy/local-path-storage.yaml

2. Bu StorageClass'ı kümenin "varsayılan (default)" depolama sınıfı yap:
kubectl patch storageclass local-path -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'

3. local-path provisioner pod'unun çalışabilmesi için (eğer Pending kalırsa) veya kontrol etmek için:
kubectl get sc
kubectl get pods -n local-path-storage

Lab vs. Gerçek Dünya (Production):

- **Bu Lab'da:** local-path kullanıyoruz; veri vm-worker-pg sunucusunun yerel diskine yazılır. Zaten tek bir PostgreSQL sunucumuz olduğu için pod her yeniden başladığında yine aynı sunucuda uyanıp kendi diskini bulur.
- **Production'da:** Sunucunun kendisi tamamen yanarsa yerel disk de gider. Bu yüzden gerçek dünyada küme içine **Azure Disk CSI Driver** veya bare-metal ortamlarda **Longhorn / Rook-Ceph** kurulur; böylece diskler ağ üzerinden çoğaltılır ve sunucu ölse bile başka sunucuya takılabilir.

BÖLÜM 6: GitHub + Helm Yapısı ile Servislerin (Redis, Postgres, Mongo) Kurulumu

1. Helm Kurulumu (vm-master üzerinde)

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

2. GitHub Reponda Oluşturacağın Dosya Yapısı

Kendi bilgisayarında bir GitHub reposu aç (veya direkt vm-master üzerinde oluşturup GitHub'a pushla) ve şu 3 values dosyasını oluştur.

(Not: 2 GB RAM'li makinelerde çalıştığımız için resources limitlerini ve Taint/Label eşleşmelerini özel olarak tanımlıyoruz).

A) helm-values/redis-values.yaml (vm-worker-infra için)

YAML

```
architecture: standalone
auth:
  enabled: true
  password: "RedisStrongPassword123!"
master:
  nodeSelector:
    tier: infra
  tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "infra"
      effect: "NoSchedule"
  persistence:
    enabled: true
    size: 2Gi
  resources:
    requests:
      memory: 128Mi
      cpu: 100m
    limits:
      memory: 512Mi
      cpu: 400m
```

B) helm-values/postgres-values.yaml (vm-worker-pg için)

YAML

```
auth:
  postgresPassword: "PgStrongPassword123!"
  username: "appuser"
  password: "AppDbPassword123!"
  database: "appdb"
primary:
```

```
  nodeSelector:
    tier: db-sql
  tolerations:
    - key: "dedicated"
      operator: "Equal"
      value: "db-sql"
      effect: "NoSchedule"
  persistence:
    enabled: true
    size: 4Gi
  resources:
    requests:
      memory: 256Mi
      cpu: 150m
    limits:
      memory: 800Mi
      cpu: 600m
```

C) helm-values/mongo-values.yaml (vm-worker-mongo için)

```
YAML
architecture: standalone
auth:
  rootPassword: "MongoRootPassword123!"
  usernames: ["appuser"]
  passwords: ["AppMongoPassword123!"]
  databases: ["appdb"]
nodeSelector:
  tier: db-nosql
tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "db-nosql"
    effect: "NoSchedule"
persistence:
  enabled: true
  size: 4Gi
resources:
  requests:
    memory: 256Mi
    cpu: 150m
  limits:
    memory: 800Mi
    cpu: 600m
```

3. Servisleri Kümede Ayağa Kaldır ve Doğrula

GitHub reponu vm-master sunucusuna git clone ile çektikten sonra kurulumları başlat:

Bash
kubectl create namespace data

helm install redis bitnami/redis -f helm-values/redis-values.yaml -n data
helm install postgres bitnami/postgresql -f helm-values/postgres-values.yaml -n data
helm install mongodb bitnami/mongodb -f helm-values/mongo-values.yaml -n data

Doğrulama (Her Servis Kendi Sunucusunda mı?):

Bash
kubectl get pods -n data -o wide

Bu çıktıda **NODE** sütununa baktığında:

- redis-master-0 $\rightarrow$ **vm-worker-infra**
- postgres-postgresql-0 $\rightarrow$ **vm-worker-pg**
- mongodb-0 $\rightarrow$ **vm-worker-mongo** üzerinde çalıştığını ve kubectl get pvc -n data yazdığında tüm disklerin **Bound** olduğunu göreceksin.

BÖLÜM 7: GitLab Container Registry, Uygulama ve Dış Erişim (Ingress)

1. Uygulamanın Veritabanlarına Nasıl Bağlanacak? (Kubernetes İç DNS)

vm-worker-app üzerinde çalışacak olan kendi uygulamanın pod'u; Redis, PostgreSQL ve MongoDB'ye sunucu IP'siyle **değil**, Kubernetes'in dahili DNS (CoreDNS) adresleriyle bağlanır:

- **Redis Host:** redis-master.data.svc.cluster.local (Port: 6379)
- **PostgreSQL Host:** postgres-postgresql.data.svc.cluster.local (Port: 5432)

- **MongoDB Host:** mongodb.data.svc.cluster.local (Port: 27017)

(Format her zaman: <servis-adi>.<namespace>.svc.cluster.local şeklindedir. Paket bu adrese gittiğinde RKE2'nin Canal VXLAN ağı paketi vm-worker-app sunucusundan alıp 10.10.1.x VNet üzerinden ilgili DB sunucusundaki pod'a ulaştırır).

2. GitLab Image Pull Secret Oluşturma

GitLab'daki projenin **Settings > Repository > Deploy Tokens** (veya profilinden Personal Access Token) kısmından **read_registry** yetkili bir token al. Ardından vm-master üzerinde:

Bash
```
kubectl create namespace app

kubectl create secret docker-registry gitlab-registry-secret \
  --docker-server=registry.gitlab.com \
  --docker-username=<GITLAB_KULLANICI_ADIN_VEYA_DEPLOY_TOKEN_ADI> \
  --docker-password=<TOKEN_DEGERI> \
  -n app
```

3. Kendi Uygulamanın Helm Chart'ı ve Ingress Tanımı

GitHub reponda kendi uygulaman için hızlı bir Helm şablonu oluştur (helm create my-app) ve values.yaml (veya doğrudan deployment.yaml / ingress.yaml) içinde şu ayarları kullan:

- **imagePullSecrets:** [{ name: gitlab-registry-secret }]
- **nodeSelector:** { tier: app } (Uygulama pod'unun vm-worker-app sunucusuna gitmesini garantiler).
- **Ingress Tanımı (Domain Olmadan nip.io Hilesi):** Gerçek bir alan adın (domain) yoksa, vm-worker-app sunucusunun Public IP adresini (örneğin 20.100.50.60 olsun) **app.20.100.50.60.nip.io** şeklinde yazabilirsin. nip.io servisi bu adresi otomatik olarak o IP'ye çözümler ve Ingress Host kuralını gerçek bir domain varmış gibi test etmeni sağlar:

YAML
```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  namespace: app
spec:
  ingressClassName: nginx
  rules:
    - host: app.<APP_WORKER_PUBLIC_IP>.nip.io
      http:
        paths:
```

```
    - path: /
      pathType: Prefix
      backend:
        service:
          name: my-app-service
          port:
            number: 80
```

Tarayıcından http://app.<APP_WORKER_PUBLIC_IP>.nip.io adresine girdiğinde akış şu şekilde tamamlanır:

1. İstek doğrudan vm-worker-app sunucusunun **80** portuna çarpar.
2. Orada dinleyen **RKE2 Nginx Ingress Controller** isteği alır, Host başlığına bakar ve my-app-service üzerinden vm-worker-app içindeki uygulama pod'una iletir.
3. Uygulama pod'un arka planda VNet (VXLAN) üzerinden vm-worker-pg, vm-worker-mongo ve vm-worker-infra sunucularındaki veritabanlarıyla konuşup cevabı döner.

BÖLÜM 8: Maliyet Kontrolü (Çalışmayı Durdurma ve Silme)

Öğrenci kredini ($100) korumak için:

- **Gece Kapatıp Sabah Devam Edeceksen (Veriler Silinmesin):** Sanal makineleri işletim sistemi içinden kapatma; Azure üzerinden **Deallocate** et (böylece vCPU/RAM ücreti durur, sadece küçük SSD disk ücreti işler):
- Bash

az vm deallocate --ids $(az vm list -g rg-rke2-manual --query "[].id" -o tsv)

-
- Sabah tekrar başlatmak için deallocate yerine start yazman yeterlidir.
- **Lab Tamamen Bittiyse (Sıfır Maliyet):** Tüm VM'leri, diskleri, VNet'i ve Public IP'leri tek komutla kalıcı olarak sil:
- Bash
- az group delete --name rg-rke2-manual --yes --no-wait
-